Topics :

1️⃣ Constructor কি
2️⃣ কেন দরকার
3️⃣ Syntax
4️⃣ Types of Constructor
5️⃣ Real Life Example
6️⃣ Code Example + Dry Run
7️⃣ Important Rules
8️⃣ Interview Notes
9️⃣ Common Mistakes
🔟 Practice Problems (10টা)

1️⃣ Constructor কি? (What is Constructor)

Constructor হলো একটি special method যা object তৈরি হওয়ার সময় automatic call হয়।

👉 সহজভাবে:

Object তৈরি হলেই Constructor automatically run হয় এবং object এর initial value set করে।

📌 Java তে constructor ব্যবহার করা হয়:

- Object initialize করার জন্য
- Default value সেট করার জন্য
- Memory allocate হওয়ার পর initial setup করার জন্য

2️⃣ Real Life Example

ধরো তুমি একটি Student ID Card বানাও।

যখন নতুন student আসে তখন:

- Name দেওয়া হয়
- ID দেওয়া হয়

- Department দেওয়া হয়

এই initial information set করা = Constructor এর কাজ।

Student s1 = new Student("Abir",101,"CSE");

এখানে object তৈরি হওয়ার সাথে সাথে constructor run হয়।

3️⃣ Constructor Syntax

Basic syntax:

```
class ClassName{
  ClassName(){
    // constructor code
  }
}
```

Example:

```
class Student{
  Student(){
    System.out.println("Object Created");
  }
}
```

Main method:

```
public class Main{
  public static void main(String[] args){
    Student s1 = new Student();
  }
}
```

Output:

Object Created

👉 কারণ object তৈরি হলেই constructor call হয়।

4 Constructor এর বৈশিষ্ট্য (Characteristics)

- Constructor এর নাম class name এর মতোই হতে হবে
- Constructor এর return type নেই

✗ ভুল

void Student()

✔ ঠিক

Student()

Object তৈরি হলে automatic call হয়

Constructor overload করা যায়

---

5 Constructor Types

Java তে প্রধানত 3 ধরনের constructor আছে।

1 Default Constructor

2 No Parameter Constructor

3 Parameterized Constructor

6 Default Constructor

যদি programmer constructor না লেখে তাহলে Java automatic constructor তৈরি করে।

Example

Java code :

```
class Car{
}
public class Main{
  public static void main(String[] args){
    Car c1 = new Car();
  }
}
```

এখানে compiler নিজেই constructor বানিয়েছে।

7 No Parameter Constructor

এখানে constructor আছে কিন্তু parameter নেই।

Example:

```
class Student{
  Student(){
    System.out.println("Student object created");
  }
}
```

Main:

```
Student s1 = new Student();
```

Output
Student object created

8 Parameterized Constructor

এখানে constructor এ parameter থাকে।

Example:

```
class Student{
  String name;
  int id;
  Student(String n, int i){
    name = n;
    id = i;
  }
  void display(){
    System.out.println(name + " " + id);
  }
}
```

Main:

```
public class Main{
  public static void main(String[] args){
    Student s1 = new Student("Abir",101);
    s1.display();
  }
}
```

Output

Abir 101

9️⃣ Dry Run (Step by Step Execution)

Code:

Student s1 = new Student("Abir",101);

Steps:

1️⃣ new Student() → object তৈরি
2️⃣ memory allocate
3️⃣ constructor call

4️⃣ "Abir" → name এ assign

5️⃣ 101 → id এ assign

```
Memory:

s1
 |
 |---- name = "Abir"
 |
 |---- id = 101
```

🔟 Real Life Example (Bank Account)

```
class BankAccount{

  String name;
  int balance;
  BankAccount(String n, int b){
     name = n;
     balance = b;
  }
  void show(){
     System.out.println("Name: "+name);
     System.out.println("Balance: "+balance);
  }
}
```

Main:

```
BankAccount a1 = new BankAccount("Abir",5000);
a1.show();
```

Output

Name: Abir
Balance: 5000

1 1 Constructor Overloading

একই class এ একাধিক constructor থাকলে তাকে constructor overloading বলে।

Example:

```
class Student{
  String name;
  int id;
  Student(){
    name = "Unknown";
    id = 0;
  }
  Student(String n,int i){
    name = n;
    id = i;
  }
}
```

1 2 Important Interview Points

✓ Constructor return type নেই
✓ Object তৈরি হলে call হয়
✓ Constructor overload করা যায়
✓ Inheritance এ constructor call হয় (super)

1 3 Common Mistakes

❌ Constructor এ return type দেওয়া

void Student()

❌ Class name mismatch

student()

✓ Correct

Student()

---

1 4 Definition

📌 Constructor:

Constructor is a special method in Java that is automatically invoked when an object is created. It is used to initialize object data members.

1 5 10 Practice Problems

Problem 1

Create a class Book

Constructor দিয়ে set করো:

- title
- price

Display method দিয়ে print করো।

Problem 2

Create class Rectangle

Constructor দিয়ে set করো:

- **length**
- **width**

Method:

area()

Problem 3

Create class Student

Constructor দিয়ে set করো:

- **name**
- **id**
- **cgpa**

Problem 4

Create class Car

Constructor দিয়ে set করো:

- **brand**
- **model**
- **price**

Problem 5

Create class Circle

Constructor দিয়ে radius set করো।

Method:

area()

Problem 6

Create Employee class

Constructor:

name
salary

Problem 7

Constructor overloading use করে

Student()
Student(String name)
Student(String name,int id)

Problem 8

Create Mobile class

Constructor দিয়ে set করো:

brand
ram
price

Problem 9

Create Box class

Constructor দিয়ে set করো:

length
width
height

Method:

volume()

Problem 10

Create BankAccount

Constructor:

name
balance

Method:

deposit()
withdraw()

NOW WE EXPLAIN IT

(✓ Inheritance এ constructor call হয় (super))

Topics :

1. this keyword in constructor
2. Constructor Chaining
3. Copy Constructor
4. super constructor
5. Constructor Interview Questions
6. Real life examples
7. Syntax & Rules
8. Dry Run
9. 10 Practice Problems

1️⃣ this Keyword in Constructor

this keyword constructor এ ব্যবহৃত হয়:

1️⃣ Current class এর instance variable reference করার জন্য
2️⃣ Constructor chaining করার জন্য (this() call)

Syntax (Instance Variable)

```
class Student {
   String name;
   int id;

   Student(String name, int id) {
      this.name = name;   // instance variable set
      this.id = id;
   }

   void display() {
      System.out.println(name + " " + id);
   }
}
```

Dry run:

```
Student s1 = new Student("Abir",101);
```

```
Memory:

s1
 |---- name = "Abir"
 |---- id = 101
```

Syntax (Constructor Chaining)

```
class Student {
   String name;
   int id;
   Student() {
      this("Unknown", 0); // constructor chaining
   }
   Student(String name, int id) {
      this.name = name;
      this.id = id;
   }
   void display() {
      System.out.println(name + " " + id);
   }
}
```

Main:

```
Student s1 = new Student();
s1.display();
```

Output:

Unknown 0

Here

(default constructor call করলো parameterized constructor কে)

If we don't understand here we explain it step by step

1️⃣ Code
2️⃣ Concept
3️⃣ Step by Step Dry Run
4️⃣ Memory idea
5️⃣ Real Life Example

1️⃣ Code Example

```
class Student {
  String name;
  int id;
  Student() {
    this("Unknown", 0);   // parameterized constructor call
  }
  Student(String name, int id) {
    this.name = name;
    this.id = id;
  }
  void display() {
    System.out.println(name + " " + id);
  }
}
public class Main {
  public static void main(String[] args) {
    Student s1 = new Student();
    s1.display();
  }
}
```

```
Output

Unknown 0
```

2️⃣ এখানে আসলে কি হচ্ছে?

এই line টা দেখো:

```
Student() {
   this("Unknown", 0);
}
```

এখানে this() ব্যবহার করে same class এর অন্য constructor call করা হয়েছে।

👉 অর্থাৎ

```
Student() → Student(String name,int id)
```

এটাই Constructor Chaining।

3️⃣ Execution Flow (Step by Step)

Main method

Student s1 = new Student();

Step 1️⃣

Java object তৈরি করবে

new Student()

এটা default constructor call করবে

Student()

Step 2️⃣

Default constructor এর ভিতরে আছে

this("Unknown",0);

এটা call করবে

Student(String name,int id)

Step 3️⃣

Parameterized constructor run হবে

this.name = name
this.id = id

মানে

name = "Unknown"
id = 0

Step 4️⃣

display() call হবে

Unknown 0

4️⃣ Execution Diagram

```
Main Method
    |
    ↓
new Student()
    |
    ↓
Student()  (default constructor)
    |
    ↓
this("Unknown",0)
    |
    ↓
Student(String name,int id)
    |
    ↓
name = "Unknown"
id = 0
```

5️⃣ Memory View

```
s1
 |
 |---- name = "Unknown"
 |
 |---- id = 0
```

6️⃣ Real Life Example

ধরো একটি Student Admission System আছে।

Student create করার দুইটা উপায়:

Case 1

User সব information দেয়

Name = Abir
ID = 101

তখন system use করবে

Student("Abir",101)

Case 2

User কিছু দেয় না

তখন system automatically default value দিবে

Name = Unknown
ID = 0

তাই default constructor internally call করে:

Student("Unknown",0)

এটাই

Student() → Student("Unknown",0)

7️⃣ Why Constructor Chaining Useful?

কারণ এতে code repeat করতে হয় না।

❌ **Without chaining**

```
Student(){
   name="Unknown";
   id=0;
}
```

✓ **With chaining**

```
Student(){
   this("Unknown",0);
}
```

👉 **Cleaner code**

👉 **Less duplication**

8️⃣ **Important Rule**

this() অবশ্যই constructor এর first line হতে হবে।

❌ **Wrong**

```
Student(){
   System.out.println("Hello");
   this("Unknown",0);

}
```

✓ Correct

```
Student(){
   this("Unknown",0);

}
```

9️⃣ **Interview Line**

Constructor chaining allows one constructor to call another constructor of the same class using the this() keyword.

Now we explain it :

this() vs super() Constructor Chaining Full Diagram

topics :

1️⃣ this() কি
2️⃣ super() কি
3️⃣ Constructor chaining flow
4️⃣ Full diagram
5️⃣ Real life example
6️⃣ Dry run execution
7️⃣ Rules (very important)
8️⃣ Interview points

1️⃣ this() কি?

this() ব্যবহার করা হয় same class এর অন্য constructor call করার জন্য।

Syntax

```
this(parameters);
```

Example

```
class Student {
   Student() {
      this("Unknown");
   }
   Student(String name) {
      System.out.println(name);
```

```
    }
}
```

Flow

Student() → Student(String name)

2️⃣ super() কি?

super() ব্যবহার করা হয় parent class constructor call করার জন্য।

Syntax

super(parameters);

Example

```
class Person {
    Person() {
        System.out.println("Person constructor");
    }
}
class Student extends Person {
    Student() {
        super();
        System.out.println("Student constructor");
    }
}
```

Flow

Person() → Student()

3️⃣ Constructor Chaining Concept

Constructor chaining মানে:

Constructor একটার ভিতর থেকে আরেকটা constructor call করা।

Two types:

Same class chaining → this()
Parent class chaining → super()

4️⃣ Full Constructor Chaining Diagram

ধরো 3টা class আছে

Person
↑
Student
↑
CSEStudent

Object create:

CSEStudent obj = new CSEStudent();

Execution flow

CSEStudent()

|

↓

super() → Student()

|

↓

super() → Person()

|

↓

Person constructor runs

|

↓

Student constructor runs

|

↓

CSEStudent constructor runs

5️⃣ Full Code Example

```
class Person {
  Person() {
    System.out.println("Person Constructor");
  }
}
class Student extends Person {
  Student() {
    System.out.println("Student Constructor");
  }
}
class CSEStudent extends Student {
  CSEStudent() {
    System.out.println("CSE Student Constructor");
  }
}
public class Main {

  public static void main(String[] args) {

    CSEStudent obj = new CSEStudent();

  }

}
```

Output

Person Constructor
Student Constructor
CSE Student Constructor

6️⃣ Diagram Visualization

```
Object Creation
    |
    ↓
CSEStudent()
    |
    ↓
super()
    |
    ↓
Student()
    |
    ↓
super()
    |
    ↓
Person()
```

Execution order

1 Person()
2 Student()
3 CSEStudent()

7 this() + super() Combined Example

```
class Person {
  Person(String name) {
    System.out.println("Person: " + name);
  }
}
class Student extends Person {
  Student() {
    this("Unknown", 0);
  }
  Student(String name, int id) {
    super(name);
    System.out.println("Student ID: " + id);
  }
}
```

Main

```
Student s1 = new Student();
```

Output

```
Person: Unknown
Student ID: 0
```

8 Full Execution Flow Diagram

```
Main Method
   |
   ↓
new Student()
   |
   ↓
Student()
   |
   ↓
this("Unknown",0)
   |
   ↓
Student(String,int)
   |
   ↓
super(name)
   |
   ↓
Person(String)
```

Execution order

1 Person constructor
2 Student(String,int)
3 Student()

9️⃣ Important Rules

Rule 1

this() or super() must be first line

Wrong

```
Student(){
System.out.println("Hello");
this("Abir");
}
```

Correct

```
Student(){
this("Abir");
}
```

Rule 2

এক constructor এ this() এবং super() একসাথে use করা যায় না

Wrong

```
Student(){
this("Abir");
super();

}
```

Rule 3

Java automatically super() call করে যদি না লিখো।

🔟 Real Life Analogy

ধরো:

```
Human
↓
Student
↓
CSEStudent
```

CSEStudent create করলে:

```
Human constructor
↓
Student constructor
↓
CSEStudent constructor
```

কারণ:

Child class parent class এর উপর depend করে।

---

1 1 this() vs super() Table

Feature	this()	super()
Calls	Same class constructor	Parent class constructor
Use	Constructor chaining	Inheritance constructor
Position	First line	First line
Keyword meaning	Current object	Parent object

1 2 Interview Short Answer

Q: Difference between this() and super()

Answer:

this() is used to call another constructor of the same class,

while super() is used to call the constructor of the parent class.